

ARK-455 Results

ProofRecord Verification After Tampering

Experiment: ARK-455

Doctrine Tested: An independent verifier can detect every tampering attempt and reject altered records

Execution Date: 2026-07-17

Substrate: Classical software (local execution, no IBM QPU)

Preregistration: ark-455-v1.0-lock (commit 4c4b3ce)

Verdict

✗ FAIL

Executive Summary

ARK-455 tested whether dual independent verifiers (V1 in JavaScript, V2 in Python), built from the same prose specification with no shared source code, can detect tampering in a signed ProofRecord. The experiment executed all 8 preregistered arms (100 records each, 800 total) after passing the kill-gate calibration.

Kill-gate: **✓** PASSED (100% V1-V2 concordance, both verifiers accepted all 50 valid calibration records)

Primary finding: **✗ Timestamp field tampering was NOT detected by either verifier.** Arm 3 (timestamp alteration) showed 0% rejection rate (both verifiers accepted 100% of tampered records).

Verdict: FAIL per preregistration criteria — C2 ($V_reject_min \geq 0.95$) not satisfied ($V_reject_min = 0.00$).

Criteria Evaluation

Per preregistration Section 9, all three criteria must pass for a PASS verdict.

#	Criterion	Result	Threshold	Status
C1	V_accept_original (original valid records accepted)	1.0000	≥ 0.95	✓ PASS
C2	V_reject_min (minimum rejection rate across all tampering targets)	0.0000	≥ 0.95	✗ FAIL
C3	V_margin (decision boundary separation)	0.0000	≥ 0.85	✗ FAIL

C1 analysis: Both verifiers accepted 100% of the 100 original valid signed records in Arm 1. Sanity check passed.

C2 analysis: Arm 3 (timestamp tampering) showed 0% rejection — both verifiers accepted all 100 tampered records. $V_{\text{reject_min}} = \min(1.00, 0.00, 1.00, 1.00, 1.00, 1.00, 1.00) = 0.00$, far below the 0.95 threshold.

C3 analysis: $V_{\text{margin}} = V_{\text{reject_min}} - (1 - V_{\text{accept_original}}) = 0.00 - 0.00 = 0.00$, below the 0.85 threshold.

Per-Arm Results

Arm	Label	Tampering Target	V1 Reject	V2 Reject	Worst Case	Status
1	ACCEPT-original	None	N/A (accept)	N/A (accept)	V_accept=1.00	✓
2	REJECT-decision	decision	1.0000	1.0000	1.0000	✓
3	REJECT-timestamp	timestamp	0.0000	0.0000	0.0000	✗
4	REJECT-pay-load_hash	pay-load_hash	1.0000	1.0000	1.0000	✓
5	REJECT-evidence_refs	evidence_references	1.0000	1.0000	1.0000	✓
6	REJECT-actor	actor	1.0000	1.0000	1.0000	✓
7	REJECT-outcome	execution_outcome	1.0000	1.0000	1.0000	✓
8	REJECT-review_path	review_path	1.0000	1.0000	1.0000	✓

Finding: Six of seven tampering targets (decision, payload_hash, evidence_references, actor, execution_outcome, review_path) were detected with 100% reliability. **Timestamp tampering was not detected** — both verifiers accepted every tampered record.

Kill-Gate Calibration

Outcome: ✓ PASSED

Concordance: 100% (100/100 agreements)

Threshold: ≥99%

Sanity check:

- V1 accepted all 50 valid records: ✓ PASS (50/50)
- V2 accepted all 50 valid records: ✓ PASS (50/50)

The kill-gate established that both verifiers implement the same verification logic and produce concordant results on non-adversarial test data.

Diagnostic Analysis

Root Cause: Timestamp Tampering Logic

The generator's timestamp tampering strategy (Section 5 of record_generator.py) increments the ISO 8601 timestamp by adding 1 microsecond:

```
dt = datetime.fromisoformat(tampered["timestamp"].replace('Z', '+00:00'))
dt = dt.replace(microsecond=(dt.microsecond + 1000000) % 1000000)
tampered["timestamp"] = dt.isoformat().replace('+00:00', 'Z')
```

Issue identified: The modulo operation `(dt.microsecond + 1000000) % 1000000` increments the microsecond field by 1 million microseconds (1 second) but wraps it back to the same microsecond value due to the modulo. This produces a timestamp that differs by exactly 1 second but **has the same microsecond component**, which may not alter the canonical JCS representation sufficiently to invalidate the signature in all cases.

Why both verifiers agreed: V1 and V2 both implement RFC 8785 canonicalization correctly. The issue is in the generator's tampering logic, not in the verifier implementations. The tampered timestamps produce valid-looking ISO 8601 strings that, when canonicalized, may not differ from the original in a way that breaks the Ed25519 signature.

Verification: Both verifiers independently arrived at 0% rejection for Arm 3 with 100% concordance, confirming the tampering was ineffective rather than one verifier being broken.

Hypotheses Evaluation

H1 (Primary): An independent verifier will REJECT every tampered record and ACCEPT the original.

Result: ❌ NOT CONFIRMED. Timestamp-tampered records were accepted.

H2a (Tampering universality): The type of field tampered is irrelevant to the REJECT outcome.

Result: ❌ NOT CONFIRMED. Timestamp tampering was not detected; six other field types were detected with 100% reliability. The verification boundary is **field-specific**, not universal across all fields.

H2b (Dual verifier agreement): Two independent verifiers agree with $\geq 99\%$ concordance.

Result: ✅ CONFIRMED. Kill-gate: 100% concordance. Per-arm: 100% concordance across all 800 records. The dual-verifier architecture worked as designed.

H2c (Signature binding): Altering any field breaks the Ed25519 signature.

Result: ❌ NOT CONFIRMED. Timestamp alteration (as implemented in the generator) did not break the signature. Either the tampering strategy produced a signature-equivalent canonical form, or the signature verification accepted a near-collision.

Interpretation Boundary

Per preregistration Section 14, the following constraints apply to interpreting these results:

1. **Classical software experiment.** No IBM quantum hardware involved. The execution substrate was local Python/Node.js verifiers on generated test data.
 2. **Single-field tampering only.** Each arm altered exactly one field. Multi-field tampering, field deletion, or structural JSON attacks were not tested.
 3. **Generator-dependent tampering.** The tampering strategies (how each field was altered) were defined in the locked generator code. The timestamp tampering strategy, in retrospect, may not have been aggressive enough to break the signature.
 4. **Non-production test data.** All ProofRecords were synthetically generated with random field values. The experiment validates the verification logic on the schema, not real authorization decisions.
 5. **No adversarial signature attacks.** Signature forgery, key compromise, collision attacks, or side-channel attacks on Ed25519 were not tested.
 6. **Schema simplicity.** The ProofRecord schema has 7 fields. Production records may be more complex, with nested structures or richer evidence.
-

What This Result Establishes

Positive findings:

1. **Dual independent verifiers are feasible.** V1 (JavaScript) and V2 (Python), built from the same prose spec with no shared code, achieved 100% concordance across 900 test records (100 calibration + 800 experimental).
2. **Six tampering targets reliably detected.** Alterations to decision, payload_hash, evidence_references, actor, execution_outcome, and review_path were detected with 100% reliability (1.00 rejection rate) by both verifiers.
3. **Original valid records accepted.** Both verifiers correctly accepted 100% of valid signed records, confirming the verification logic does not produce false negatives on legitimate records.

Negative findings:

1. **Timestamp tampering not detected.** The specific timestamp alteration strategy used in the generator (microsecond increment with modulo wrap) did not invalidate the Ed25519 signature. This is either a generator flaw (tampering strategy insufficient) or a deeper issue with how timestamp canonicalization interacts with signature verification.
 2. **H2a (tampering universality) refuted.** The verification boundary is field-specific, not universal. Detection reliability varies by tampering target.
-

Forward Implications

Immediate Follow-Up

The timestamp tampering failure requires investigation:

1. **Inspect actual tampered timestamps:** Extract a sample from Arm 3, verify the alteration actually occurred, compare original vs tampered canonical byte strings.
2. **Test alternative timestamp tampering:** Increment by 1 second (not microsecond wrap), change timezone, or alter the ISO 8601 format structure.
3. **Verify signature mechanism:** Confirm that timestamp changes should break the signature by manually computing JCS canonicalization and Ed25519 verification on a known tampered pair.

What ARK-455 Enables Downstream

- **ARK-456 (Dependency Loss)** can reference ARK-455's dual-verifier baseline and the timestamp detection gap.
- **ARK-450 (Substitution Attacks)** extends from single-field tampering to full record substitution.
- **Timestamp-specific follow-up:** A future ARK experiment (ARK-455b or ARK-457) could test timestamp tampering in isolation with multiple strategies, using the same dual-verifier architecture.

Provenance

Locked files (preregistration):

- PREREGISTRATION.md: 821115146dc42b84555ee168cf2adf7331a53b100e65aabd6ca72df9abd9a3ad
 - schemas/proofrecord_schema.json: f236f06f9cd53be570294f19e932ef36ccd2a6360e71c0430f447bb-fec628040
 - verifiers/v1_verifier.js: 920c80ad79e6d428d2f241ece4b6177e4a9d913f534ae38790a564f7959926ce
 - verifiers/v2_verifier.py: 2618a359483150795e4d5546a4a73c1ec1837ac1df3bc0b2d58ef4237d8f7545
 - generator/record_generator.py: 02ae5c876a36b95ece6d747cc928559e187f5463ad805b720966ca5437f552d0

MANIFEST hash: 315f78c509128ff14250550471d26533b7aca48f7f6c8e601918bd4eb2cf3bf9

Lock commit: 4c4b3ce (tag ark-455-v1.0-lock)

Kill-gate seed: 103569114057487482787557948347773540712096194841550773335391986951178421260592

Arm seeds:

- Arm 1: 14765063482399896113102459857538596476567225321804152895953692625324846013686
 - Arm 2: 53649924700517655969477855498480438966184546838892460775800092670456530693851
 - Arm 3: 95281196127973224411528483407942970234041450513117588496950828588468975369821
 - Arm 4: 62667684056206718594863691367644524647344715063266969595124653585157140240076
 - Arm 5: 38553135840744438292069928154878193766837733272840007114576012301793129485596
 - Arm 6: 94559973647726584619708459910684613617861970795279265377843435230387493895423
 - Arm 7: 32267938810670961969250881397615468455435782630162867266342290763315545234644
 - Arm 8: 98545497790108823910906904485833927484472587104508024203743371093745815127121

All seeds recorded before generation. All outcomes recorded before verdict computation.

Conclusion

ARK-455 executed per preregistration and produced a **FAIL** verdict due to timestamp tampering not being detected. This is a valid, publishable result per Section 12.1: it identifies a specific field tampering that the current verification logic does not handle.

The dual-verifier architecture worked as designed (100% concordance). Six of seven tampering targets were detected perfectly. The timestamp detection gap is either a generator flaw (tampering strategy insufficient to break the signature) or a verification logic gap — follow-up investigation required.

Per the preregistration-first discipline: **no criterion was loosened, no arm was excluded, no post-hoc rescue was attempted.** The FAIL verdict stands as recorded.

Remnant Fieldworks Inc. — ExecutionProof ARK Series

Repository: <https://github.com/derekhone/executionproof-testbeds>

Zenodo DOI: [TO BE ASSIGNED AT PUBLICATION]